

Waste Classification API

Deep Learning • FastAPI • Docker • Kubernetes

- A Real-Time Deep Learning Deployment for Environmental Impact

By Ramya Subramanian Porselva Bharathi



Problem Statement & Objective

This project aims to automate waste classification using deep learning, making recycling smarter and scalable.

Environmental Urgency

Global waste production is rising rapidly, expected to reach **3.4 billion tons by 2050**. Mismanaged waste contributes to **pollution, health hazards, and climate change**.

Circular Economy Challenge

Circular economies (e.g., **TOMRA's mission**) rely on **efficient waste sorting and recycling**. Manual sorting is **slow, error-prone, and labor-intensive**.

The Need for Smart Waste Sorting

AI-powered solutions can **automate the classification** of waste at recycling centers. Improves **recycling efficiency, reduces contamination**, and promotes **sustainability**.

Role of Machine Learning

Machine Learning models can **classify waste images like** plastic, glass, metal, etc. Enables real-time predictions for **automated waste segregation systems**.

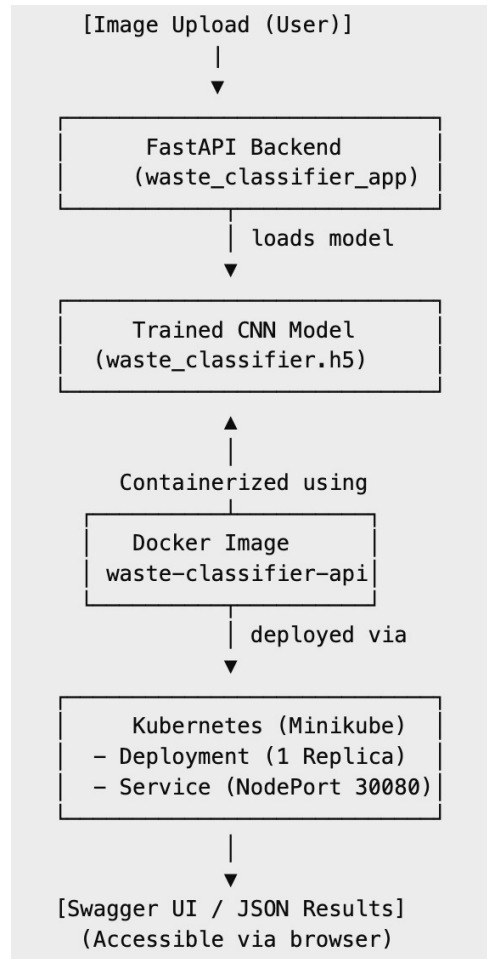
Technical Overview

- **Python** - A Core programming language used for model development and API integration.
- **TensorFlow & Keras**- Used to build and train the convolutional neural network (CNN) for image classification.
- **FastAPI** - A Lightweight, high-performance web framework used to serve the ML model via REST API.
- **Docker** - Containerized the ML application to ensure portability and environment consistency.
- **Kubernetes (Minikube)** - Used for local container orchestration and scaling, simulating real-world deployment scenarios.
- **NumPy** - Used for handling image arrays and numerical operations.
- **Pillow** - Used to preprocess uploaded image files before feeding them into the ML model.
- **Uvicorn** - A lightning-fast ASGI server used to deploy the FastAPI app locally and within the Docker container.
- **Git & GitHub for version control**

Project Overview

- - Trained CNN model to classify waste images (plastic, paper, glass, etc.)
 - - Served model via FastAPI for real-time predictions
 - - Containerized with Docker
 - - Deployed using Kubernetes (Minikube)
 - - Exposed REST API via Swagger UI
- 

Architecture Diagram



Dataset Summary

Dataset: Garbage Classification Dataset (Kaggle)

Categories (6 classes): Cardboard, Plastic, Metal, Glass, Paper, Trash

Directory Structure:

- Images were categorized into folders by class to facilitate model training

Image Size:

- All images resized to **150 × 150 pixels**

Preprocessing Techniques:

- Rescaling pixel values (0–255 → 0–1)
- Normalization using ImageDataGenerator

Data Split:

- 80% Training set
- 20% Validation set

Tool Used for Preprocessing:

- ImageDataGenerator from Keras

Model Architecture

CNN Architecture - A **Convolutional Neural Network (CNN)** was designed to classify waste into 6 categories.

Architecture includes:

3 Convolutional Layers (Conv2D): Extract spatial features using filters.

MaxPooling Layers after each Conv layer: Reduce dimensionality and preserve key features.

Flatten Layer: Converts 3D output to 1D for the dense layer.

Dense Layer: Learns high-level feature patterns.

Softmax Output Layer: Provides class probabilities for 6 waste categories.

Training Configuration

Optimizer: Adam

Loss Function: Categorical Cross-Entropy

Activation Functions: ReLU (hidden layers), Softmax (output layer)

Output: The final model outputs a **6-class probability distribution** (cardboard, glass, metal, paper, plastic, trash).

Input (150x150x3)

↓

Conv2D → ReLU → MaxPooling

↓

Conv2D → ReLU → MaxPooling

↓

Conv2D → ReLU → MaxPooling

↓

Flatten → Dense → Softmax (6 Classes)

Training & Evaluation

Model Training Summary

Number of Epochs: Trained for 30 epochs with early stopping

Batch Size: 32

Accuracy & Loss Curves

Training and validation accuracy steadily improved across epochs.

Loss decreased consistently with minimal overfitting.

Class Mapping Output

```
print("Class indices:", train_data.class_indices)
```

Output:

```
{'cardboard': 0, 'glass': 1, 'metal': 2, 'paper': 3, 'plastic': 4, 'trash': 5}
```

Output: waste_classifier.h5 (saved model)

Class labels saved in class_names.txt

FastAPI Integration



- Built a RESTful API using FastAPI that accepts image uploads via multipart/form-data



- Endpoint: /predict accepts image files



- Swagger UI auto-generated at /docs



- Returns predicted class and confidence score

Docker Containerization

- Defines app environment using Dockerfile.
- Copies app and model directories into the container.
- Installs dependencies from requirements.txt.
- Runs Uvicorn server on port 8000.
- Installs TensorFlow, FastAPI, and other dependencies.
- Docker image is built using: `docker build -t waste-classifier-api .`

Output of docker images:

```
(.venv) (base) ramya@Ramyas-MacBook-Air waste_classifier_app % docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
waste-classifier-api	latest	065137901c3d	3 days ago	2.47GB
waste-classifier-app	latest	05597ce20bbc	6 days ago	2.33GB

Dockerfile content

```
# Use an official Python runtime as a base image
FROM python:3.11-slim

# Set the working directory
WORKDIR /app

# Copy only necessary files
COPY ./app /app/app
COPY ./model /app/model
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# Expose port
EXPOSE 8000

# Command to run the app
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

RUN pip install python-multipart
```

Kubernetes Deployment

deployment.yaml:

- 1 replica
- Container port: 8000
- imagePullPolicy: Never

service.yaml:

- Exposes NodePort on 30080

Minikube:

- Used to host the app locally

```
(.venv) (base) ramya@Ramyas-MacBook-Air waste_classifier_app % kubectl apply -f deployment.yaml
deployment.apps/waste-api-deployment created

(.venv) (base) ramya@Ramyas-MacBook-Air waste_classifier_app % kubectl apply -f service.yaml
service/waste-api-service created

(.venv) (base) ramya@Ramyas-MacBook-Air waste_classifier_app % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
waste-api-deployment-657b976bcd-nlhj 1/1     Running   0           16s

(.venv) (base) ramya@Ramyas-MacBook-Air waste_classifier_app % minikube service waste-api-service
|-----|-----|-----|-----|
| NAMESPACE | NAME           | TARGET PORT | URL                               |
|-----|-----|-----|-----|
| default   | waste-api-service | 8000        | http://192.168.49.2:30080       |
|-----|-----|-----|-----|
🔗 Starting tunnel for service waste-api-service.
|-----|-----|-----|-----|
| NAMESPACE | NAME           | TARGET PORT | URL                               |
|-----|-----|-----|-----|
| default   | waste-api-service |             | http://127.0.0.1:64664         |
|-----|-----|-----|-----|
🔗 Opening service default/waste-api-service in default browser...
! Because you are using a Docker driver on darwin, the terminal needs to be open to run it.
```

Live API Demo

FastAPI - Swagger UI

127.0.0.1:64664/docs#/

☆

FastAPI

0.1.0

OAS 3.1

[/openapi.json](#)

default

^

POST

/predict

Predict

⌵

Schemas

^

Body_predict_predict_post > Expand all object

HTTPValidationError > Expand all object

ValidationError > Expand all object

Live API Demo

FastAPI - Swagger UI

127.0.0.1:64664/docs#/default/predict_predict_post

POST

/predict

Predict

Parameters

No parameters

Cancel

Reset

Request body

required

multipart/form-data

file

required

string(\$binary)

Choose file

plastic99.jpg

Execute

Clear

Responses

Curl

curl -X 'POST' \n'http://127.0.0.1:64664/predict' \n-H 'accept: application/json' \n-H 'Content-Type: multipart/form-data' \n-F 'file=@plastic99.jpg;type=image/jpeg'

Request URL

http://127.0.0.1:64664/predict

Server response

Code

Details

200

Response body

{\n "predicted_class": "plastic",\n "confidence": 0.75\n}

Download

Response headers

content-length: 47\ncontent-type: application/json\ndate: Fri, 04 Jul 2025 17:03:19 GMT\nserver: uvicorn

Responses

Learnings & Challenges

Learnings

- Gained hands-on experience building **RESTful APIs** with **FastAPI**
- Learned how to **build Docker images** and run containers locally
- Understood **Kubernetes fundamentals**: Deployments, Pods, Services
- Showcased **end-to-end ML model deployment** with **Docker** and **Kubernetes**
- Developed skills in debugging container issues:
 - **ImagePullBackOff**: Fixed using Minikube's Docker and imagePullPolicy: Never
 - **Docker container crash**: Resolved by adding python-multipart to requirements.txt
 - **Prediction consistency**: Fixed class mismatch by aligning training class indices with prediction mapping
- Gained real-world relevance to **TOMRA** and **sustainability** by building an eco-friendly waste classification system
- **Hands-on experience** with **model training**, **APIs**, **CI**, and **microservices** through the entire workflow

Challenges

- **Class label mismatch**: Ensured class labels in predictions matched those from class_names.txt
- **Container networking issues in Minikube**: Resolved service reachability by correcting port mappings and ensuring pods were running

Final Summary & GitHub Link

- - Built and deployed full-stack waste classification API
- - Stack: Python, TensorFlow, FastAPI, Docker, Kubernetes
- - Accessible via REST API and documented with Swagger UI
- GitHub: <https://github.com/RamyaSP6/waste-classifier-api>



THANK YOU!

